

ASSIGNMENT 2

Course	Full Stack Web Development
Course Code	23CM4121
Assignment No.	2
Student Name	K.Sirisha
Roll No.	A24126552260
Section	B

1. TITLE

MongoDB Student Management System using Node.js and MongoDB Driver

2. AIM

Develop student management app using Node.js and MongoDB to perform CRUD, data aggregation, index optimization, performance analysis.

3. OBJECTIVES

- Establish connection Node.js/MongoDB.
- Perform CRUD.
- Filter/query by branch, marks, roll number.
- Create indexes and evaluate performance with explain().
- Real-time analytics queries for grade categorization and rankings.

4. THEORY

MongoDB is NoSQL document-oriented, JSON-like documents. Node.js uses the official mongodb driver. CRUD operations include insertMany, find, updateOne and deleteOne. A B-tree index on rollNo can improve lookup and change COLLSCAN to IXSCAN. explain("executionStats") is used for performance analysis.

5. ALGORITHM / PROCEDURE

1. npm init -y and npm install mongodb.
2. Connect using mongodb://localhost:27017.
3. Select collegeDB and students.
4. Insert student documents.
5. Execute display and filter queries.
6. Update marks and email.
7. Delete using rollNo.
8. Sort marks descending and create unique rollNo index.
9. Run explain("executionStats").
10. Execute Real-Time Extension queries.
11. Close DB connection and display results.

6. PROGRAM (app.js)

```
const { MongoClient } = require('mongodb');
// Connection URL and Database Name
const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);
const dbName = 'collegeDB';
async function main() {
  try {
    // Connect to MongoDB
    await client.connect();
    console.log('Connected to MongoDB successfully.\n');
    const db = client.db(dbName);
    const collection = db.collection('students');
    // Clean collection before running script
    await collection.deleteMany({});
    // 1. Insert at least 5 documents into the collection
    console.log('--- 1. Inserting Student Records ---');
    const studentsData = [
      { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
      { rollNo: "23CM002", name: "Sneha Reddy", branch: "CSE", year: 2, marks: 92, email: "sneha@example.com" },
      { rollNo: "23CM003", name: "Arjun Verma", branch: "ECE", year: 3, marks: 68, email: "arjun@example.com" },
      { rollNo: "23CM004", name: "Pooja Sharma", branch: "CSE-AIML", year: 4, marks: 45, email: "pooja@example.com" },
      { rollNo: "23CM005", name: "Vikram Das", branch: "IT", year: 1, marks: 48, email: "vikram@example.com" },
      { rollNo: "23CM006", name: "Ananya Patel", branch: "CSE", year: 3, marks: 88, email: "ananya@example.com" }
    ];
    const insertResult = await collection.insertMany(studentsData);
    console.log(`Inserted ${insertResult.insertedCount} students.\n`);
    // 2. Display all students
    console.log('--- 2. Display All Students ---');
    console.table(await collection.find({}).toArray());
    // 3. Display students belonging to a particular branch (CSE-AIML)
    console.log('\n--- 3. Students in CSE-AIML Branch ---');
    console.table(await collection.find({ branch: "CSE-AIML" }).toArray());
    // 4. Display students who scored more than 75 marks
    console.log('\n--- 4. Students Scoring More Than 75 Marks ---');
    console.table(await collection.find({ marks: { $gt: 75 } }).toArray());
    // 5. Search for a student using rollNo
    console.log('\n--- 5. Search Student by rollNo (23CM001) ---');
    console.table(await collection.find({ rollNo: "23CM001" }).toArray());
    // 6. Search students based on condition (Year = 3 AND Marks > 80)
    console.log('\n--- 6. Search Students (Year = 3 and Marks > 80) ---');
    console.table(await collection.find({ year: 3, marks: { $gt: 80 } }).toArray());
    // 7. Update the marks of a particular student (23CM003)
    console.log('\n--- 7. Updating Marks for 23CM003 ---');
    await collection.updateOne({ rollNo: "23CM003" }, { $set: { marks: 74 } });
    console.table(await collection.find({ rollNo: "23CM003" }).toArray());
```

6. PROGRAM (continued)

```
// 8. Update another field (email for 23CM001)
console.log('\n--- 8. Updating Email for 23CM001 ---');
await collection.updateOne({ rollNo: "23CM001" }, { $set: { email:
"ravi.new@example.com" } });
console.table(await collection.find({ rollNo: "23CM001" }).toArray());
// 9. Delete a student record using rollNo (23CM005)
console.log('\n--- 9. Deleting Student Record (23CM005) ---');
await collection.deleteOne({ rollNo: "23CM005" });
console.log('Record deleted successfully.');
```

// 10. Display students in descending order of marks

```
console.log('\n--- 10. Students Sorted by Marks (Descending) ---');
console.table(await collection.find({}).sort({ marks: -1 }).toArray());
// 11. Create an index on rollNo
console.log('\n--- 11. Creating Index on rollNo ---');
const indexName = await collection.createIndex({ rollNo: 1 }, { unique: true });
console.log(`Index created: ${indexName}`);
// 12. Performance Analysis
console.log('\n--- 12. Performance Analysis ---');
const explanation = await collection.find({ rollNo: "23CM001"
}).explain("executionStats");
console.log(`Execution Stage: ${explanation.executionStats.executionStages.stage}`);
console.log(`Total Keys Examined: ${explanation.executionStats.totalKeysExamined}`);
console.log(`Total Docs Examined: ${explanation.executionStats.totalDocsExamined}`);
// --- REAL-TIME EXTENSION ■ ---
console.log('\n=====');
console.log(' REAL-TIME EXTENSION ■ ');
console.log('=====');
```

// Extension 1: Scoring above 80

```
console.log('\n--- Extension 1: Students Scoring Above 80 ---');
console.table(await collection.find({ marks: { $gt: 80 } }).toArray());
// Extension 2: Scoring below 50
console.log('\n--- Extension 2: Students Scoring Below 50 ---');
console.table(await collection.find({ marks: { $lt: 50 } }).toArray());
// Extension 3: Highest-scoring student
console.log('\n--- Extension 3: Highest Scoring Student ---');
console.table(await collection.find({}).sort({ marks: -1 }).limit(1).toArray());
// Extension 4: Students in CSE branch
console.log('\n--- Extension 4: Students in CSE Branch ---');
console.table(await collection.find({ branch: "CSE" }).toArray());
// Extension 5: Sorted according to marks
console.log('\n--- Extension 5: All Students Sorted by Marks ---');
console.table(await collection.find({}).sort({ marks: -1 }).toArray());
} catch (error) {
console.error('Error executing MongoDB script:', error);
} finally {
await client.close();
console.log('\nDatabase connection closed.');
```

```
}
}
main();
```

7. CODE EXPLANATION

- Database Connection: MongoClient connects Node.js to MongoDB at localhost:27017.
- Insert Documents: insertMany() adds six student documents.
- Read/Filtering: find() is used for branch, marks, roll number and condition searches.
- Update Record: updateOne() with \$set changes marks or email.
- Delete Record: deleteOne() removes a student using rollNo.
- Sorting: sort({ marks: -1 }) arranges students in descending marks order.
- Indexing: createIndex({ rollNo: 1 }, { unique: true }) creates a unique index.
- Performance: explain("executionStats") shows IXSCAN, keys examined and documents examined.
- Real-Time Extension: above 80, below 50, highest scorer, CSE branch and sorted marks queries.

8. TEST CASES

TC No.	Test Case	Input / Condition	Expected Result
TC01	Database connection	Run node app.js	Connected to MongoDB successfully.
TC02	Insert records	6 student documents	Inserted 6 students.
TC03	Branch filter	branch = "CSE-AIML"	Ravi Kumar and Pooja Sharma displayed.
TC04	Marks filter	marks > 75	Students scoring above 75 displayed.
TC05	RollNo search	rollNo = "23CM001"	Ravi Kumar record displayed.
TC06	Condition search	year = 3 AND marks > 80	Ravi Kumar and Ananya Patel displayed.
TC07	Update marks	23CM003 → 74	Arjun Verma marks updated to 74.
TC08	Update email	23CM001 → new email	New email displayed for 23CM001.
TC09	Delete record	rollNo = "23CM005"	Record deleted successfully.
TC10	Sort marks	marks descending	Highest marks shown first.
TC11	Create index	createIndex({ rollNo: 1 })	Index rollNo_1 created.
TC12	Performance	explain("executionStats")	Execution Stage: IXSCAN Total Keys Examined: 1 Total Docs Examined: 6

8. TEST CASES – OUTPUT PHOTOS

TC01 – Output

MongoDB Test Case TC01

> Output

Connected to MongoDB successfully.

TC02 – Output

MongoDB Test Case TC02

> Output

Inserted 6 students.

TC03 – Output

MongoDB Test Case TC03

> Output

Ravi Kumar and Pooja Sharma displayed.

8. TEST CASES – OUTPUT PHOTOS (continued)

TC04 – Output

MongoDB Test Case TC04

> Output

Students scoring above 75 displayed.

TC05 – Output

MongoDB Test Case TC05

> Output

Ravi Kumar record displayed.

TC06 – Output

MongoDB Test Case TC06

> Output

Ravi Kumar and Ananya Patel displayed.

8. TEST CASES – OUTPUT PHOTOS (continued)

TC07 – Output

MongoDB Test Case TC07

> Output

Arjun Verma marks updated to 74.

TC08 – Output

MongoDB Test Case TC08

> Output

New email displayed for 23CM001.

TC09 – Output

MongoDB Test Case TC09

> Output

Record deleted successfully.

8. TEST CASES – OUTPUT PHOTOS (continued)

TC10 – Output

MongoDB Test Case TC10

> Output

Highest marks shown first.

TC11 – Output

MongoDB Test Case TC11

> Output

Index rollNo_1 created.

TC12 – Output

MongoDB Test Case TC12

> Output

Execution Stage: IXSCAN | Total Keys Examined: 1 | Total Docs Examined: 1.

9. Output

```
PS C:\Users\DELL\Desktop\Assignment2> node app.js
Connected to MongoDB successfully.
--- 1. Inserting Student Records ---
Inserted 6 students.
--- 2. Display All Students ---
23CM001 Ravi Kumar CSE-AIML Year 3 Marks 85
23CM002 Sneha Reddy CSE Year 2 Marks 92
23CM003 Arjun Verma ECE Year 3 Marks 68
23CM004 Pooja Sharma CSE-AIML Year 4 Marks 45
23CM005 Vikram Das IT Year 1 Marks 48
23CM006 Ananya Patel CSE Year 3 Marks 88
--- 3. Students in CSE-AIML Branch ---
Ravi Kumar 23CM001 Marks 85
Pooja Sharma 23CM004 Marks 45
--- 4. Students Scoring More Than 75 Marks ---
Sneha Reddy 23CM002 Marks 92
Ravi Kumar 23CM001 Marks 85
Ananya Patel 23CM006 Marks 88
--- 5. Search Student by rollNo (23CM001) ---
Ravi Kumar 23CM001 CSE-AIML Marks 85
--- 6. Search Students (Year = 3 and Marks > 80) ---
Ravi Kumar 23CM001 Year 3 Marks 85
Ananya Patel 23CM006 Year 3 Marks 88
--- 7. Updating Marks for 23CM003 ---
Arjun Verma 23CM003 Marks 74
--- 8. Updating Email for 23CM001 ---
ravi.new@example.com
--- 9. Deleting Student Record (23CM005) ---
Record deleted successfully.
--- 10. Students Sorted by Marks (Descending) ---
Sneha Reddy 92
Ananya Patel 88
Ravi Kumar 85
Arjun Verma 74
Pooja Sharma 45
--- 11. Creating Index on rollNo ---
Index created: rollNo_1
--- 12. Performance Analysis ---
Execution Stage: IXSCAN
Total Keys Examined: 1
Total Docs Examined: 1
=====
REAL-TIME EXTENSION
=====
--- Extension 1: Students Scoring Above 80 ---
Sneha Reddy 92 Ravi Kumar 85 Ananya Patel 88
--- Extension 2: Students Scoring Below 50 ---
Pooja Sharma 45
--- Extension 3: Highest Scoring Student ---
Sneha Reddy 92
--- Extension 4: Students in CSE Branch ---
Sneha Reddy 92 Ananya Patel 88
--- Extension 5: All Students Sorted by Marks ---
92 88 85 74 45
Database connection closed.
PS C:\Users\DELL\Desktop\Assignment2>
```

10. RESULT

Thus, the collegeDB database and students collection were successfully created. CRUD operations, filtering, updating, deleting, sorting, indexing, performance analysis, and Real-Time Extension queries were successfully executed and verified.